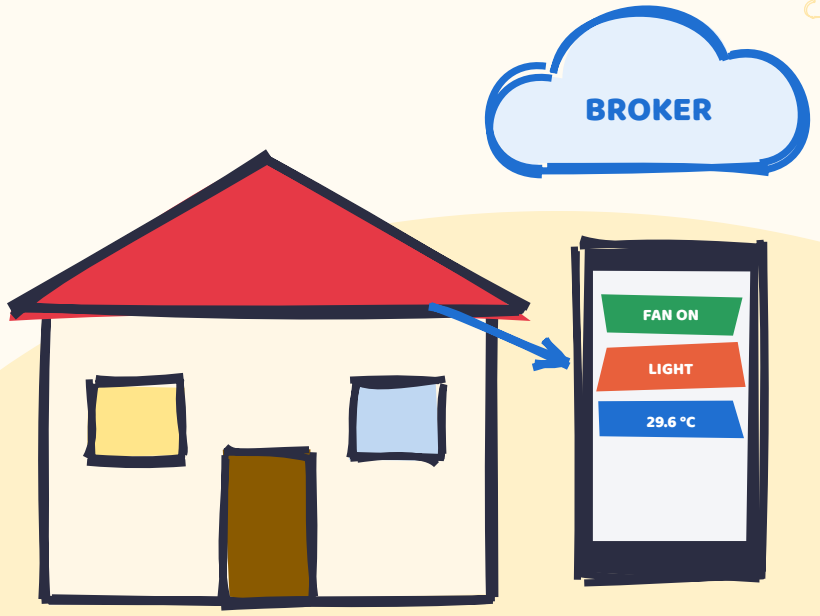
Tech Leaders · Class 6

Smart Home Cloud Control Hub

Switch it from anywhere, automate it, and make it survive failure

MQTT pub/sub | Relays | Rules + override



Did you know?
Billions of smart devices use MQTT, a protocol designed in 1999 to monitor oil pipelines over slow satellite links.

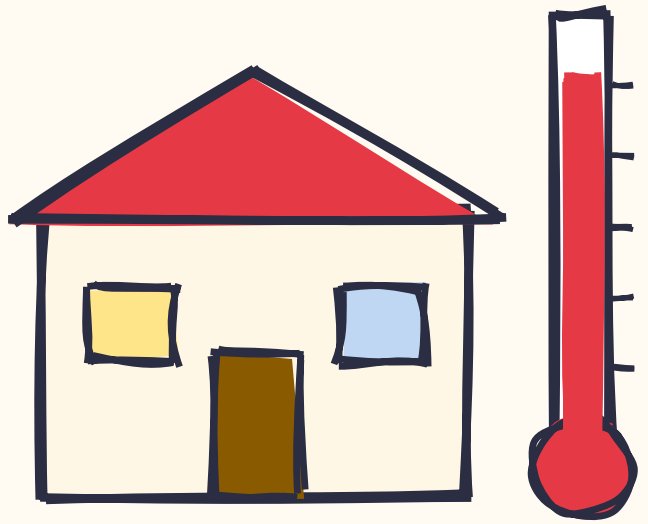
LEARN

The engineering problem

It's 40 °C and you're not home

A real scenario

You are stuck in traffic. The house is baking, your grandmother is home, and the fan is off. A smart home could **switch it on from your phone**, or better, **switch it on by itself**. But what happens if the internet is down?



Call home?

Phone app?

Automatic rule!

Today: you build a hub you can control from your phone, that **automates itself** and keeps working **when the cloud fails**.

Learning objectives

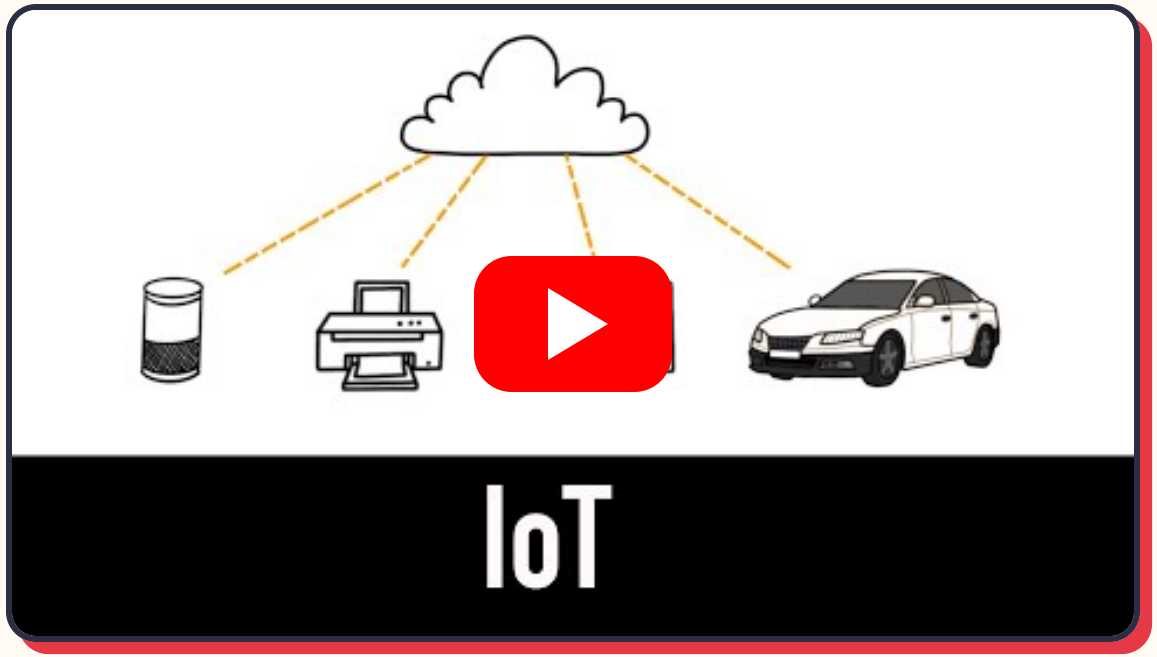
By the end of this class I can...

1	**Publish / subscribe:** explain how **MQTT** moves messages through a broker using **topics**	**Proof:** draw the path of a 'fan on' tap from phone to relay
2	**Relays:** use a **relay** to let a 3.3 V pin switch a separate circuit, safely	**Proof:** wire a relay and explain why we only switch 5 V loads
3	**Cloud code:** connect the ESP32 to Wi-Fi and MQTT, **publish** sensor data and **subscribe** to commands	**Proof:** your phone shows the temperature and switches a relay
4	**Automation:** write rules with **hysteresis** and a **manual override**	**Proof:** fan on above 30 °C, off below 28 °C, and a tap always wins
5	**Design for failure:** keep the home working when the **internet goes down**	**Proof:** unplug the router: rules still run, dashboard shows offline

LEARN

Watch and think

Video time!



Internet of Things (IoT) Explained in 3 Minutes

Connected Cookie · about 4 minutes · youtu.be/-M74HobvzGA

Watch for...

1. What makes a device part of the **Internet of Things**?
2. Where does the data go?
3. Name one risk of connected devices.

Scan to watch

After the video: Sketch our smart home hub: which devices talk to it, and what does the cloud do?

LEARN

Objective 1 · Publish / subscribe

Publish / subscribe through a broker

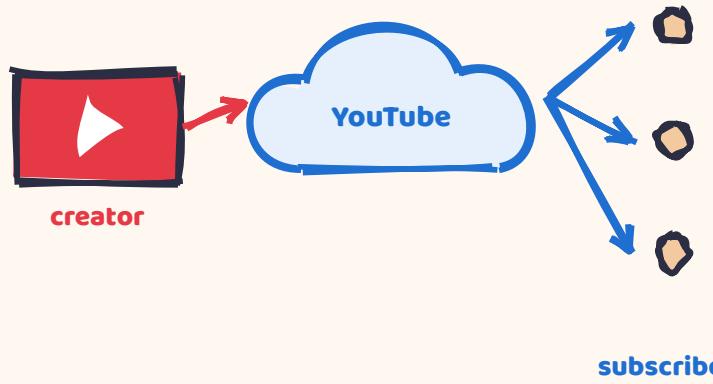
The idea

In **MQTT**, devices never talk to each other directly. They talk to a **broker** (a server).
Publish = send a message to a **topic**.
Subscribe = ask the broker for every message on a topic.

The publisher does not know or care who is listening.

Both ways
Our phone publishes 'cmd'. The ESP32 publishes 'temp'. Both are publishers AND subscribers.

Example: subscribing on YouTube



A creator **publishes** a video to a channel. YouTube is the **broker**. Everyone who **subscribed** gets it, and the creator never needs to know their names. A channel is a **topic**.

LEARN

Objective 1 · Publish / subscribe

Topics are addresses with levels

The idea

A topic is a text path split by **/**. Good design makes it easy to subscribe to exactly what you need:

`pakturk/team7/temp`

`pakturk/team7/cmd/relay/0`

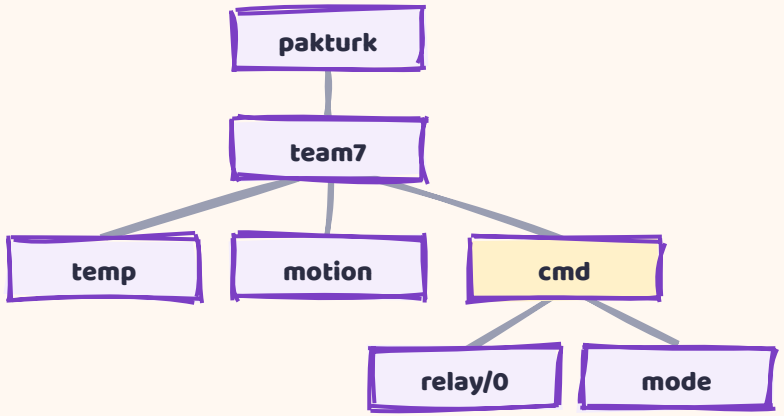
+ matches one level, **#** matches everything below:

`pakturk/team7/cmd/#`

Public = visible

We use a free public broker. Anyone can read pakturk/#. Never publish names or locations.

Example: a postal address



Pakistan / Islamabad / F-7 / Street 6 / House 12. Each level narrows it down. A postman for all of F-7 subscribes to **Pakistan/Islamabad/F-7/#**.

Objective 2 · Relays

A relay: small signal, separate circuit

The idea

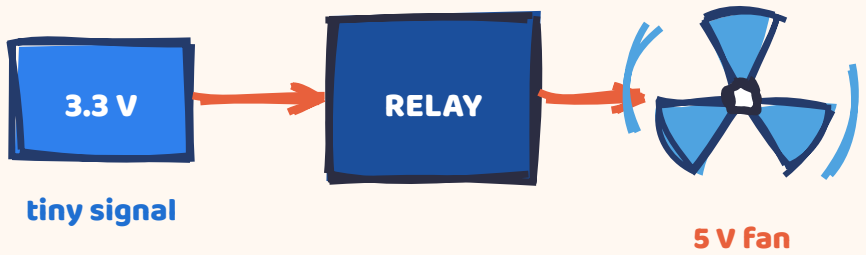
An ESP32 pin gives 3.3 V and only a few milliamps. A **relay** is an electrically controlled switch: a small current through a coil pulls a metal contact closed in a **completely separate** circuit.

Our module is **active LOW**: writing **LOW turns it ON**. Every relay should start OFF (HIGH) in setup().

Listen

The click you hear is the contact moving. No click = no switching.

Example: a car key



Turning a small key lets a tiny current reach the starter relay. The relay then connects the battery to the starter motor, which needs **hundreds of amps**. Your hand never touches that current.

LEARN

Objective 3 · Cloud code

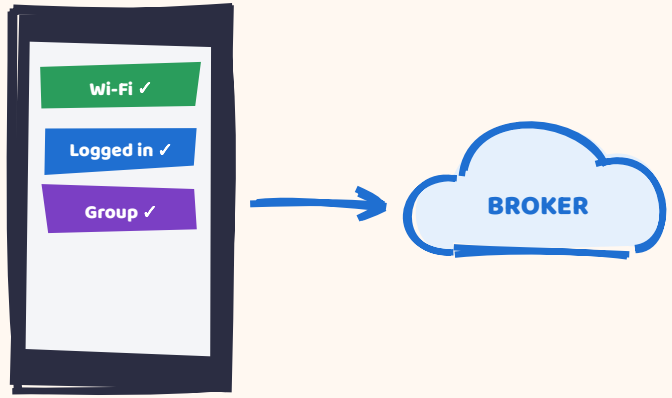
Three connections, in order

The idea

1. **Wi-Fi**: join the network.
2. **MQTT connect**: log in to the broker with a unique client ID.
3. **Subscribe** to command topics, then call **mqtt.loop()** constantly so messages are delivered.

If any link drops, **reconnect from that step**, without freezing the rest of the program.

Example: joining a class WhatsApp group



Your phone needs **internet** (Wi-Fi), then **the app logged in** (connect), then you must **be in the group** (subscribe). If you close the app, messages wait: that is why **mqtt.loop()** must keep running.

Objective 4 · Automation

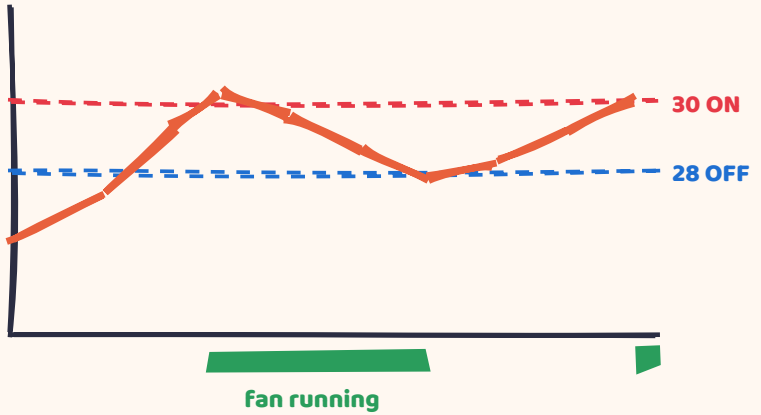
Hysteresis: two thresholds, not one

The idea

A rule like **'fan on if over 30 °C, else off'** fails at 29.9, 30.1, 29.9... The fan clicks on and off every few seconds, wearing out the relay.

Fix it with a **gap**: ON above **30**, OFF below **28**. Between 28 and 30, **do nothing**: keep the current state.

Example: an air conditioner



A home AC set to 24 °C does not switch off at 23.9 and on at 24.1. It runs until about **23**, rests until about **25**, then starts again. That gap protects the compressor and saves electricity.

LEARN

Objective 4 · Automation

A human must always be able to take over

The idea

Automation is helpful until it is wrong. A **manual override** means a person's command beats the rules.

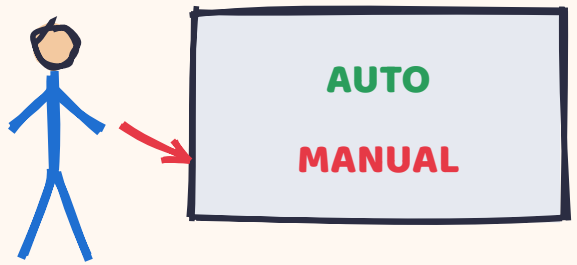
Our design: tapping any relay on the phone sets **manual = true**, and the rules pause. Tapping **AUTO** hands control back.

Priority order: **safety → human → rules**.

Debate

Challenge: should manual mode switch back to AUTO by itself after 30 minutes? Argue both sides.

Example: aircraft autopilot



Autopilot flies most of a journey, but the moment a pilot moves the controls firmly, the plane **obeys the pilot**. A system that fought the pilot would be dangerous.

LEARN

Objective 5 · Design for failure

Run the rules at the edge, not in the cloud

The idea

If the **cloud** decides 'fan on', the home is helpless when the internet fails. If the **ESP32** decides (the **edge**), the rules keep working offline. The cloud is only for **remote control and viewing**.

Also: never let a reconnect loop **block** the rules.

Design test

Ask of every smart product: what still works with the Wi-Fi off?

Example: a smart lock that would not open



There have been real cases where cloud servers went down and owners could not unlock their own smart locks or switch on lights. Good designs keep a **local way** to work.

LEARN

Objective 5 · Design for failure

Last Will: report your own disappearance

The idea

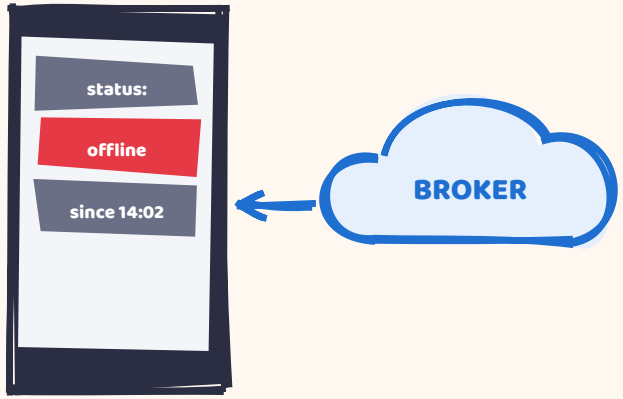
When the ESP32 connects, it gives the broker a **Last Will**: topic **status**, message **offline**, retained. If the ESP32 vanishes (power cut, crash), the broker publishes it **on its behalf**.

On connect it publishes **online** itself. So the dashboard is always truthful.

Retained

Retained = the broker keeps the last value, so a phone that opens later still sees it.

Example: an out-of-office auto-reply



Before a holiday you set an auto-reply. While you are away, the **email server answers for you**: 'I am not available'. The Last Will is the broker answering for a device that cannot.

LEARN

Think ahead

Our hub has a big security hole

The uncomfortable truth

Anyone who guesses **pakturk/team7/cmd/relay/0** can switch your fan. The public broker has **no username, no password, no encryption**. Our hub is a demo, not a product.

How real products fix it

Private broker with a password per device · TLS encryption (port 8883) · topic permissions (each device may only use its own topics) · no default passwords.

In **TL-13 Securing an IoT Device** you will attack this hub (with permission) and fix it. Keep your code!

Part 2 · Make it

Now let's put it to work!

You know the ideas. Time to play, build and test. Use these slides while you work:

1. We only switch 5 V. Here's why.
2. Kit per group
3. Wire relays, PIR and DHT22
4. Build the phone dashboard
5. Connect: Wi-Fi, broker, Last Will
6. Receive commands and the override
7. Publish data and run the rules

Objective 2 · Relays · safety

We only switch 5 V. Here's why.

No mains, ever. 230 V AC can kill. Relay boards in class have exposed pins. Only licensed electricians wire mains.

5 V loads only: USB LED lamps and a 5 V fan. Same skills, zero danger.

Separate power: the fan gets its own 5 V supply, not the ESP32's 3V3 pin.

Check before power: a partner inspects wiring. Unplug before changing anything.

Real homes

Real smart plugs are sealed and certified. That is the product version of what we build.

Remember

A design that is unsafe is not a smart design.

Build it

Kit per group



ESP32
+ USB cable



4-ch relay
active LOW module



PIR sensor
HC-SR501, motion



DHT22
temperature + humidity



2 × 5 V LED lamps
+ 5 V USB fan



Phone or laptop
MQTT dashboard app

Libraries: PubSubClient (Nick O'Leary), **DHT sensor library**. **Broker:** broker.hivemq.com, port 1883 (public, free, no password).

Topic prefix
Each team gets a unique prefix, e.g. pakturk/team7/. Two teams with the same prefix control each other's relays!

MAKE

Objective 2 · Build step 1 of 3

Wire relays, PIR and DHT22

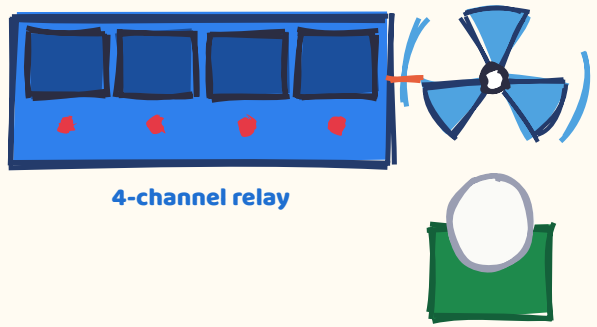
a Relay IN1 to IN4 to **GPIO 16, 17, 18, 19**. GND to GND.

b Relay module power: VCC to **3V3** and JD-VCC to **5 V** (remove the jumper). Fan and lamps switch through COM and NO.

c PIR OUT to **GPIO 27**, VCC to 5 V (VIN), GND to GND. Its output is 3.3 V, safe for the ESP32.

d DHT22 OUT to **GPIO 4**, + to 3V3.

e Load circuit: separate 5 V supply → COM; NO → fan + ; fan - → supply -.



Check: With everything OFF, the relay LEDs are dark and nothing clicks at power-up.

If relays never switch off: a 5 V module can't see 3.3 V as HIGH. Use the JD-VCC split or a 3.3 V relay module.

MAKE

Objective 1 · Build step 2 of 3

Build the phone dashboard

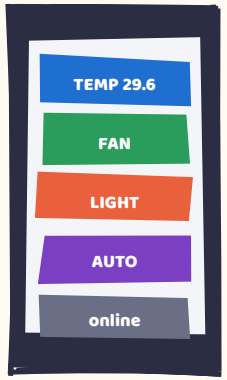
a Install an MQTT dashboard app (e.g. **IoT MQTT Panel** on Android) or use the **HiveMQ browser client** on a laptop.

b Add a connection: broker.hivemq.com, port 1883, any client ID.

c Add **gauges** subscribed to **.../temp** and **.../motion**, and a text tile for **.../status**.

d Add 4 **switches** that publish 1 / 0 to **.../cmd/relay/0** to **.../relay/3**, and a button that publishes **auto** to **.../cmd/mode**.

e Before any code, publish a test message from the app and see it arrive in a second client.



Check: A message you publish appears in the other client within a second.

Each switch must publish to cmd/... and read state from relay/... so the button always shows the real state.

Objective 3 · Code part 1

Connect: Wi-Fi, broker, Last Will

```
#include <WiFi.h>
#include <PubSubClient.h>
#include <DHT.h>
const char* SSID = "SchoolLab";
const char* PASS = "********";
#define T "pakturk/team7/"              // UNIQUE prefix!
const int RELAY[4] = {16, 17, 18, 19};  // active LOW
#define PIR 27
WiFiClient net;  PubSubClient mqtt(net);
DHT dht(4, DHT22);
bool manual = false;                    // override flag
unsigned long lastTry = 0, lastPub = 0;

void setRelay(int i, bool on) {
  digitalWrite(RELAY[i], on ? LOW : HIGH);    // LOW = ON
  mqtt.publish((String(T) + "relay/" + i).c_str(), on ? "1" : "0", true);
}
void keepConnected() {                  // never blocks for long
  if (mqtt.connected() || millis() - lastTry < 5000) return;
  lastTry = millis();
  if (WiFi.status() != WL_CONNECTED) { WiFi.reconnect(); return; }
  String id = "hub-" + String((uint32_t)ESP.getEfuseMac(), HEX);
  // Last Will: broker says "offline" for us if we vanish
  if (mqtt.connect(id.c_str(), T "status", 1, true, "offline")) {
    mqtt.publish(T "status", "online", true);
    mqtt.subscribe(T "cmd/#");
  }
}
```

One try every 5 s

keepConnected() returns at once if it is not time. The old **while(!connected)** loop would freeze the rules (O5).

Last Will (O5)

connect(id, willTopic, QoS, retain, willMsg). Then we publish **online** ourselves.

Topics (O1)

T "status" joins two strings at compile time: pakturk/team7/status.

Objective 3 + 4 · Code part 2

Receive commands and the override

```
void onMessage(char* topic, byte* payload, unsigned int len) {
  String t = topic, msg;
  for (unsigned int i = 0; i < len; i++) msg += (char)payload[i];

  if (t == T "cmd/mode") manual = (msg != "auto");

  for (int i = 0; i < 4; i++) {
    if (t == String(T "cmd/relay/") + i) {
      manual = true;            // a human touched it: rules pause
      setRelay(i, msg == "1");
    }
  }
}

void setup() {
  for (int i = 0; i < 4; i++) {
    pinMode(RELAY[i], OUTPUT);
    digitalWrite(RELAY[i], HIGH);       // all OFF at start
  }
  pinMode(PIR, INPUT);
  dht.begin();
  WiFi.begin(SSID, PASS);
  mqtt.setServer("broker.hivemq.com", 1883);
  mqtt.setCallback(onMessage);          // subscribe handler
}
```

Payload is bytes

MQTT sends raw bytes, not a String. We build **msg** one character at a time.

Override (O4)

Any relay tap sets **manual = true**. Only the AUTO button gives control back.

Safe start (O2)

All relays HIGH (OFF) before anything else. A power cut must never switch things on.

Objective 4 + 5 · Code part 3

Publish data and run the rules

```
void loop() {
  keepConnected();
  mqtt.loop();                          // deliver messages
  if (millis() - lastPub < 5000) return;
  lastPub = millis();

  float t = dht.readTemperature();
  bool motion = digitalRead(PIR);
  if (!isnan(t)) mqtt.publish(T "temp", String(t, 1).c_str());
  mqtt.publish(T "motion", motion ? "1" : "0");

  if (!manual) {                        // RULES run here, at the edge
    if (t > 30) setRelay(0, true);      // fan ON above 30
    if (t < 28) setRelay(0, false);     // fan OFF below 28
    setRelay(1, motion);                // light follows motion
  }
}
```

Hysteresis in 2 lines (O4)

Between 28 and 30 neither line runs, so the fan **keeps its state**. That gap is the whole idea.

Works offline (O5)

No line here needs the internet. publish() just fails quietly while the broker is unreachable.

NaN safety

If the DHT read fails, t is NaN and both comparisons are false: nothing changes. Safe by design.

Objectives 4 + 5 · Build step 3 of 3

Prove it works, then prove it fails well

Test	Expected	Pass?
Warm the DHT22 with your hand past 30	Fan ON within 5 s	
Let it cool between 28 and 30	Fan stays ON	
Cool below 28	Fan OFF	
Tap FAN OFF while hot	Fan OFF, stays off	
Tap AUTO	Rules resume	
Unplug the router	Fan rule still works	
Pull the ESP32's power	Dashboard: offline	

Engineer's habit
Write expected results BEFORE you test. Otherwise you just describe what happened.

The router test is the most important row. It is the difference between a gadget and a product.

Test like an engineer

Troubleshooting

Symptom	Likely cause
Never connects to MQTT	school firewall blocks port 1883: try a phone hotspot
Another team switches your fan	same topic prefix: make T unique
Relays always ON	3.3 V logic on a 5 V module: split VCC / JD-VCC
Fan clicks on and off	hysteresis missing, or relay and ESP32 share a weak supply
Dashboard button shows wrong state	button not reading relay/... state topic

Debug checklist

1. Serial-print every message received
2. Watch the topics in a second client
3. Test relay alone with a blink sketch

Use a **second MQTT client** as a spy: subscribe to **pakturk/team7/#** and you see everything your hub and phone say to each other.

MAKE

Level up!

Apply what you learned

Choose your challenge

Timer rule: lights off automatically at 22:00 using NTP time (O4).

Auto-return: manual mode switches back to AUTO after 30 minutes (O4).

Local button: a physical switch that toggles the fan even with no Wi-Fi at all (O5).

Dashboard: plot the last hour of temperature using retained or logged data (O3).

Think like an engineer

Your hub uses a public broker with no password. List what a stranger could see and do. How would you fix it?

Boss level

Connect your hub to a teammate's ESP-NOW network from TL-05 so one hub controls the whole floor.

Objective check

Can I really do it? Prove it.

01	Trace a 'fan on' tap: which device publishes, to which topic, through what, and who subscribes?	○ ○ ○
02	Why does a relay let a 3.3 V pin switch a fan? Why do we never switch 230 V in class?	○ ○ ○
03	Name the 3 connections in order. Why must **mqtt.loop()** run all the time?	○ ○ ○
04	Temperature goes 29, 31, 29.5, 28.5, 27. When is the fan on? What does a tap on FAN OFF do to the rules?	○ ○ ○
05	The internet goes down. What still works, what stops, and what does the dashboard show?	○ ○ ○

Rate yourself

Colour one circle each. Red: not yet. Amber: with help. Green: I can explain it to a friend.

Teacher: O4 answer: ON at 31, stays ON at 29.5 and 28.5, OFF at 27.

Show what you know

What did we learn?

- [x] MQTT: **publish** to a topic, the **broker** delivers to **subscribers**
- [x] A **relay** lets a tiny signal switch a separate circuit; 5 V only in class
- [x] Wi-Fi → broker → subscribe, and reconnect **without blocking**
- [x] **Hysteresis** (30 on / 28 off) and a **manual override** that always wins
- [x] Rules at the **edge** + **Last Will** = a home that fails gracefully

Exit ticket: Give one reason rules should run on the ESP32 and not in the cloud.

At home

List every smart device at home. What happens to each one when the Wi-Fi is off?



Next class: TL-07 Smart Parking Dashboard